ĐẠI HỌC QUỐC GIA HÀ NỘI

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

KHOA VẬT LÝ

THIẾT KẾ VÀ MÔ PHỎNG CÁC KHỐI CHỨC NĂNG CỦA BỘ XỬ LÝ MIPS 32-BIT ĐƠN CHU KỲ

Hà Nội – 2026

LỜI MỞ ĐẦU

Bộ xử lý đơn chu kỳ là mô hình kiến trúc máy tính trong đó mỗi lệnh được hoàn thành trong một chu kỳ xung nhịp. Mặc dù không tối ưu về hiệu năng do chu kỳ phải đủ dài để đáp ứng đường đi dữ liệu chậm nhất, kiến trúc này có ưu điểm sư phạm quan trọng: các bước nạp lệnh, giải mã, đọc thanh ghi, thực thi, truy cập bộ nhớ và ghi kết quả có thể được quan sát trong một đường đi dữ liệu thống nhất.

Báo cáo trình bày việc khảo sát và phân tích mã nguồn Verilog trong dự án Single-Cycle-Core-Hus. Phạm vi mã nguồn hiện có tập trung vào các khối nền tảng của datapath, bộ điều khiển chính, bộ giải mã ALU và các testbench tương ứng. Trên cơ sở đó, báo cáo mô tả nguyên lý hoạt động, quan hệ tín hiệu, cách kiểm thử và đánh giá mức độ hoàn thiện của thiết kế.

Lưu ý về phạm vi: gói mã nguồn được cung cấp chưa thể hiện một hệ thống CPU hoàn chỉnh có tích hợp đầy đủ instruction memory, data memory và top-level kết nối tất cả các khối. Vì vậy, báo cáo phân biệt rõ giữa những chức năng đã được hiện thực trong mã nguồn và những chức năng mới dừng ở giao tiếp hoặc cần hoàn thiện thêm; không xem các phần chưa có là kết quả đã kiểm chứng.

TÓM TẮT

Đề tài khảo sát thiết kế các thành phần của bộ xử lý MIPS 32-bit đơn chu kỳ bằng ngôn ngữ mô tả phần cứng Verilog. Các mô-đun chính gồm bộ đếm chương trình PC, mạch cộng địa chỉ, bộ dịch trái hai bit, mạch mở rộng dấu, tập thanh ghi 32 × 32-bit, ALU, các bộ ghép kênh và bộ điều khiển giải mã opcode. Datapath hỗ trợ các đường xử lý cần thiết cho nhóm lệnh R-type, lw, sw, beq, addi và j ở mức tín hiệu điều khiển/đường dữ liệu.

Các testbench được cung cấp kiểm tra riêng bộ điều khiển chính, các khối hỗ trợ datapath và một số hoạt động cơ bản của datapath. Tuy nhiên, kiểm thử hiện tại chủ yếu là kiểm thử chức năng theo kịch bản cố định; chưa có kiểm tra tự động đầy đủ cho kết quả mong đợi của mọi lệnh, chưa có kiểm thử riêng ALU/ALU decoder và chưa chứng minh được toàn bộ CPU hoạt động đầu-cuối. Báo cáo đề xuất các bước hoàn thiện nhằm nâng độ tin cậy và khả năng trình diễn của thiết kế.

Từ khóa: MIPS, single-cycle, Verilog, datapath, control unit, ALU, register file, testbench, mô phỏng số.

MỤC LỤC

TÓM TẮT 2

MỤC LỤC 4

1. GIỚI THIỆU ĐỀ TÀI 6

1.1. Bối cảnh 6

1.2. Mục tiêu 6

1.3. Phạm vi và giả định 6

1.4. Phương pháp thực hiện 7

2. CƠ SỞ LÝ THUYẾT 7

2.1. Kiến trúc MIPS 32-bit 7

2.2. Nguyên lý xử lý đơn chu kỳ 7

2.3. Datapath và Control Unit 8

2.4. Các phép toán ALU 8

3. PHÂN TÍCH KIẾN TRÚC VÀ CÁC MÔ-ĐUN 8

3.1. Sơ đồ khối ở mức chức năng 8

3.2. Thanh ghi PC và logic cập nhật địa chỉ 8

3.3. Tập thanh ghi (Register File) 9

3.4. Mạch mở rộng dấu (Sign Extension) 9

3.5. Bộ dịch trái hai bit và bộ cộng 10

3.6. Bộ ghép kênh MUX 2:1 10

3.7. ALU 10

3.8. Bộ điều khiển chính (Main Control) 10

3.9. Bộ giải mã ALU (ALU Decoder) 11

3.10. Quan hệ giữa các khối 11

4. PHÂN TÍCH ĐƯỜNG ĐI DỮ LIỆU THEO TỪNG LỆNH 11

4.1. Lệnh R-type 11

4.2. Lệnh addi 12
4.3. Lệnh lw 12
4.4. Lệnh sw 12
4.5. Lệnh beq 12
4.6. Lệnh j 12
4.7. Bảng tổng hợp đường xử lý 13
5. THIẾT KẾ VÀ ĐÁNH GIÁ KIỂM THỬ 13
5.1. Môi trường và phương pháp kiểm thử 13
5.2. Kiểm thử Main Control 13
5.3. Kiểm thử các khối hỗ trợ 13
5.4. Kiểm thử Datapath 14
5.5. Ma trận kiểm thử đề xuất 14
5.6. Tiêu chí đánh giá 14
6. ĐÁNH GIÁ THIẾT KẾ 15
6.1. Ưu điểm 15
6.2. Hạn chế và vấn đề kỹ thuật cần xử lý 15
6.3. Nhận xét về mức độ hoàn thiện ..... **Error! Bookmark not defined.**
7. ĐỀ XUẤT HOÀN THIỆN VÀ PHÁT TRIỂN 16
7.1. Hoàn thiện top-level CPU 16
7.2. Bổ sung mô hình bộ nhớ 16
7.3. Chuẩn hóa register file 16
7.4. Nâng cấp verification 16
7.5. Mở rộng tập lệnh 17
8. KẾT LUẬN 17
TÀI LIỆU THAM KHẢO 18
PHỤ LỤC A. DANH MỤC TỆP MÃ NGUỒN ĐÃ KHẢO SÁT 18

PHỤ LỤC B. KỊCH BẢN TRÌNH BÀY MIỆNG GỢI Ý .**Error! Bookmark not defined.**

1. GIỚI THIỆU ĐỀ TÀI

1.1. Bối cảnh

CPU là thành phần trung tâm của hệ thống máy tính, thực hiện việc diễn giải và thi hành các lệnh chương trình. Để hiểu cách một CPU vận hành, cần phân tích hai thành phần có quan hệ chặt chẽ: datapath và control unit. Datapath thực hiện các phép toán và vận chuyển dữ liệu; control unit giải mã lệnh và phát sinh tín hiệu điều khiển để lựa chọn đường xử lý phù hợp.

Kiến trúc MIPS thường được sử dụng trong giảng dạy vì tập lệnh có cấu trúc tương đối đều, định dạng lệnh rõ ràng và cách tổ chức datapath thuận tiện cho việc minh họa. Trong mô hình đơn chu kỳ, một lệnh đi qua toàn bộ các khối chức năng trong một chu kỳ.

1.2. Mục tiêu

Phân tích cấu trúc và nguyên lý hoạt động của các mô-đun Verilog trong dự án.
Giải thích đường đi dữ liệu cho các nhóm lệnh cơ bản: R-type, lw, sw, beq, addi và j.
Làm rõ vai trò của các tín hiệu điều khiển và mối liên hệ giữa main control, ALU decoder và datapath.
Đánh giá các testbench hiện có, chỉ ra giới hạn kiểm thử và các điểm cần hiệu chỉnh.
Đề xuất hướng tích hợp, xác minh và mở rộng để tiến tới một CPU MIPS đơn chu kỳ hoàn chỉnh.

1.3. Phạm vi và giả định

Phân tích dựa trên các tệp: datapath.v, alu.v, regfile.v, các mô-đun hỗ trợ, main_control.v, alu_decoder, các tệp ALU ở thư mục Alu Core và ba testbench. Không có tệp mô tả bộ nhớ lệnh, bộ nhớ dữ liệu hoặc một mô-đun top-level CPU hoàn chỉnh trong cấu trúc mã nguồn được cung cấp. Tệp mips_top.v chỉ chứa một đoạn khởi tạo ALU, không phải một module hoàn chỉnh.

1.4. Phương pháp thực hiện

Phương pháp gồm: (i) đọc và phân loại mã nguồn theo chức năng; (ii) đối chiếu cấu trúc tín hiệu với datapath MIPS đơn chu kỳ chuẩn; (iii) phân tích các đường xử lý lệnh; (iv) đánh giá testbench theo tiêu chí stimulus, expected result và assertion; (v) xác định thiếu sót và đề xuất kế hoạch hoàn thiện. Các nhận định về kết quả mô phỏng chỉ được đưa ra trong giới hạn nội dung testbench thể hiện.

2. CƠ SỞ LÝ THUYẾT

2.1. Kiến trúc MIPS 32-bit

MIPS là kiến trúc tập lệnh kiểu RISC. Trong mô hình đang xét, thanh ghi dữ liệu và ALU có độ rộng 32 bit; địa chỉ lệnh tăng 4 byte giữa hai lệnh tuần tự. Các lệnh được mã hóa trong từ 32 bit. Những trường quan trọng gồm opcode [31:26], rs [25:21], rt [20:16], rd [15:11], shamt [10:6], funct [5:0] và immediate [15:0].

Trường lệnh	Bit	Ý nghĩa
opcode	[31:26]	Xác định nhóm lệnh và cách giải mã chính.
rs	[25:21]	Chỉ số thanh ghi nguồn thứ nhất.
rt	[20:16]	Thanh ghi nguồn thứ hai hoặc thanh ghi đích tùy lệnh.
rd	[15:11]	Thanh ghi đích đối với nhiều lệnh R-type.
immediate	[15:0]	Hằng số/độ lệch 16 bit, thường được mở rộng dấu.
funct	[5:0]	Xác định phép toán cụ thể trong nhóm R-type.

2.2. Nguyên lý xử lý đơn chu kỳ

Một lệnh điển hình trải qua các bước: nạp lệnh tại địa chỉ PC; giải mã opcode; đọc các thanh ghi nguồn; chọn toán hạng cho ALU; thực hiện phép toán hoặc tính địa chỉ; truy cập data memory nếu cần; chọn dữ liệu ghi về register file; cập nhật PC. Các bước diễn ra trong cùng một chu kỳ, dù các khối tổ hợp hoạt động nối tiếp về mặt thời gian lan truyền.

Thời gian chu kỳ tối thiểu bị giới hạn bởi đường đi dữ liệu có độ trễ lớn nhất. Với tập lệnh cơ bản, lệnh lw thường tạo đường đi dài: PC, instruction memory, register file, ALU tính địa chỉ, data memory, mux write-back, register file. Đây là nhược điểm hiệu năng điển hình của kiến trúc đơn chu kỳ.

2.3. Datapath và Control Unit

Datapath bao gồm các phần tử lưu trữ và xử lý dữ liệu như PC, register file, ALU, bộ cộng, sign extension, shift-left và các mux. Control unit nhận opcode và phát sinh các tín hiệu như RegWrite, RegDst, ALUSrc, Branch, MemWrite, MemToReg, ALUOp và Jump. ALU decoder kết hợp ALUOp với trường funct để tạo mã điều khiển ALU.

2.4. Các phép toán ALU

ALUControl	Phép toán	Mô tả
000	AND	AND từng bit giữa hai toán hạng.
001	OR	OR từng bit giữa hai toán hạng.
010	ADD	Cộng hai toán hạng 32 bit.
110	SUB	Trừ toán hạng B khỏi toán hạng A.
111	SLT	So sánh nhỏ hơn có dấu; kết quả 1 nếu A < B, ngược lại 0.

Cờ Zero được đặt bằng 1 khi kết quả ALU bằng 0. Trong lệnh beq, hai toán hạng được trừ; nếu bằng nhau thì hiệu bằng 0 và Zero = 1. Tín hiệu chọn nhánh thường được hình thành bởi Branch AND Zero. Trong mã nguồn hiện tại, datapath nhận pcsrc từ bên ngoài, do đó phép AND này chưa được tích hợp ngay trong datapath.

3. PHÂN TÍCH KIẾN TRÚC VÀ CÁC MÔ-ĐUN

3.1. Sơ đồ khối ở mức chức năng

Theo datapath.v, luồng dữ liệu có thể mô tả như sau: PC, Instruction Memory , instr , Register File/Sign Extension/Control; Register File , MUX ALUSrc , ALU; ALU , Data Memory (bên ngoài) và MUX MemToReg , Register File; đồng thời PC đi qua các mạch cộng và mux để chọn PC+4, địa chỉ nhánh hoặc địa chỉ jump.

Các giao tiếp instr và readdata được khai báo ở datapath, nhưng bộ nhớ tương ứng không được hiện thực trong tệp này. Vì vậy, đây là một datapath có giao diện kết nối bộ nhớ, chưa phải toàn bộ hệ thống CPU.

3.2. Thanh ghi PC và logic cập nhật địa chỉ

Mô-đun flopr được tham số hóa độ rộng, sử dụng cạnh lên clk để chốt dữ liệu và reset bất đồng bộ mức cao để đưa đầu ra về 0. Trong datapath, flopr #(32) được dùng làm PC. Khi

reset được kích hoạt, PC = 0; sau khi reset được nhả, PC nhận pcnext tại cạnh lên tiếp theo.

Mạch pcadd1 tính PCPlus4 = PC + 4. Đối với lệnh tuần tự, PC tiếp theo bằng PCPlus4. Đối với nhánh có điều kiện, địa chỉ nhánh được tính theo công thức PCBranch = (PC + 4) + (SignImm << 2). Với lệnh jump, địa chỉ đích được ghép theo dạng {PCPlus4[31:28], instr[25:0], 2'b00}. Hai mux liên tiếp lựa chọn giữa PCPlus4/PCBranch và sau đó lựa chọn giữa kết quả trên với PCJump.

Việc dịch trái 2 bit phản ánh đặc tính địa chỉ lệnh căn hàng 4 byte. Trường immediate sau mở rộng dấu biểu diễn độ lệch theo đơn vị word; nhân 4 chuyển độ lệch thành byte trước khi cộng vào PC+4.

3.3. Tập thanh ghi (Register File)

Mô-đun regfile khai báo mảng rf gồm 32 phần tử, mỗi phần tử 32 bit. Hai cổng đọc rd1 và rd2 là bất đồng bộ, tức dữ liệu đầu ra thay đổi theo địa chỉ đọc mà không cần chờ cạnh clock. Cổng ghi hoạt động đồng bộ ở cạnh lên: nếu we3 = 1 thì rf[a3] nhận wd3.

Thanh ghi $zero được xử lý bằng logic đọc: khi a1 hoặc a2 bằng 0, đầu ra tương ứng luôn bằng 0. Đây là cách bảo đảm giá trị đọc của $zero không đổi ngay cả khi vùng rf[0] chưa được khởi tạo. Tuy nhiên, mã hiện tại vẫn cho phép ghi vào rf[0]; việc ghi này không làm thay đổi giá trị quan sát qua cổng đọc, nhưng nên chặn rõ ràng bằng điều kiện a3 != 0 để phản ánh đúng đặc tả và tránh trạng thái nội bộ không cần thiết.

Một điểm quan trọng khác là các thanh ghi rf[1] đến rf[31] chưa được reset hoặc khởi tạo. Trong mô phỏng, trước khi ghi giá trị xác định vào một thanh ghi, việc đọc thanh ghi đó có thể cho giá trị X. Testbench cần khởi tạo trạng thái hoặc ghi các thanh ghi cần thiết trước khi sử dụng.

3.4. Mạch mở rộng dấu (Sign Extension)

Mô-đun signext chuyển immediate 16 bit thành 32 bit bằng cách sao chép bit dấu a[15] vào 16 bit cao: y = {{16{a[15]}}, a}. Nếu immediate không âm, 16 bit cao bằng 0; nếu immediate âm theo biểu diễn bù hai, 16 bit cao bằng 1. Ví dụ 16'h000A được mở rộng thành 32'h0000000A; 16'hFFF4 được mở rộng thành 32'hFFFFFFF4.

3.5. Bộ dịch trái hai bit và bộ cộng

Mô-đun sl2 thực hiện y = {a[29:0], 2'b00}, tương đương dịch trái logic hai bit trong độ rộng 32 bit. Mô-đun adder thực hiện phép cộng 32 bit. Hai khối này được dùng trong tính địa chỉ nhánh và cập nhật PC. Vì kết quả giữ độ rộng 32 bit, phần carry vượt quá bit 31 bị loại bỏ theo số học modulo 2^32.

3.6. Bộ ghép kênh MUX 2:1

mux2 được tham số hóa độ rộng và thực hiện y = s ? d1 : d0. MUX xuất hiện tại nhiều vị trí: chọn thanh ghi đích (rt/rd), chọn toán hạng B của ALU (register/immediate), chọn dữ liệu write-back (ALU/data memory), chọn PC tuần tự hay nhánh, và chọn PC jump. Việc sử dụng một mô-đun mux tổng quát giúp giảm lặp mã và thống nhất hành vi lựa chọn.

3.7. ALU

ALU nhận hai toán hạng 32 bit a, b và mã điều khiển 3 bit. Khối sử dụng always @(*) cùng case để mô tả logic tổ hợp. Các phép AND, OR, ADD, SUB và SLT được hiện thực. Phép SLT sử dụng $signed(a) và $signed(b), do đó so sánh theo số nguyên có dấu bù hai. Đầu ra zero được gán bằng biểu thức (result == 32'b0).

Thiết kế hiện chưa có phép toán dịch, XOR, NOR hay phép so sánh không dấu. Đây không nhất thiết là lỗi nếu phạm vi đề tài chỉ yêu cầu tập lệnh cơ bản, nhưng cần nêu rõ giới hạn tập lệnh được hỗ trợ.

3.8. Bộ điều khiển chính (Main Control)

main_control giải mã opcode [31:26] và phát sinh các tín hiệu điều khiển. Các opcode được định nghĩa gồm R-type = 000000, lw = 100011, sw = 101011, beq = 000100, addi = 001000 và j = 000010. Với mỗi opcode, khối gán đồng thời một vector tín hiệu theo thứ tự {reg_write, reg_dst, alu_src, branch, mem_write, mem_to_reg, alu_op[1:0], jump}. Nhánh default đưa toàn bộ tín hiệu về 0, là lựa chọn an toàn hơn so với giữ giá trị cũ trong mạch tổ hợp.

Lệnh	RegWrite	RegDst	ALUSrc	Branch	MemWrite	MemToReg	ALUOp	Jump
R-type	1	1	0	0	0	0	10	0
lw	1	0	1	0	0	1	00	0
sw	0	0	1	0	1	0	00	0
beq	0	0	0	1	0	0	01	0

Lệnh	RegWrite	RegDst	ALUSrc	Branch	MemWrite	MemToReg	ALUOp	Jump
addi	1	0	1	0	0	0	00	0
j	0	0	0	0	0	0	00	1

Diễn giải: R-type ghi kết quả ALU vào rd; lw ghi dữ liệu đọc từ bộ nhớ vào rt; sw không ghi register file mà phát lệnh ghi bộ nhớ; beq kích hoạt điều kiện nhánh; addi cộng immediate với rs và ghi vào rt; j chọn địa chỉ jump cho PC.

3.9. Bộ giải mã ALU (ALU Decoder)

ALU decoder nhận ALUOp và funct. Với ALUOp = 00, ALUControl được đặt là ADD, phù hợp cho tính địa chỉ lw/sw và phép cộng addi. Với ALUOp = 01, ALUControl là SUB, phù hợp cho phép so sánh beq. Với ALUOp = 10, trường funct xác định phép toán R-type: ADD (100000), SUB (100010), AND (100100), OR (100101) và SLT (101010).

Trong mã, các nhánh casez lần lượt là 2'b00, 2'b?1 và 2'b1?. Với các giá trị ALUOp hợp lệ 00, 01, 10, kết quả giải mã đúng theo ý đồ. Tuy nhiên, mẫu ?1 cũng khớp với 11 và được đặt trước mẫu 1?, vì vậy ALUOp = 11 sẽ bị giải mã thành SUB. Nên thay cách viết bằng các giá trị cụ thể hoặc quy định rõ các trạng thái không sử dụng để tránh nhập nhằng.

3.10. Quan hệ giữa các khối

Main Control không trực tiếp tạo ALUControl; nó tạo ALUOp. ALU decoder dùng ALUOp và funct để tạo ALUControl đưa đến ALU. Main Control đồng thời cung cấp các tín hiệu điều khiển mux và ghi bộ nhớ/ghi thanh ghi. Datapath trả cờ Zero về phía điều khiển. Để tạo tín hiệu pcsrc cho datapath, hệ thống tích hợp cần logic pcsrc = branch AND zero. Trong mã hiện tại, pcsrc là đầu vào độc lập, do đó logic kết hợp này cần được bổ sung ở cấp top-level.

4. PHÂN TÍCH ĐƯỜNG ĐI DỮ LIỆU THEO TỪNG LỆNH

4.1. Lệnh R-type

Đối với R-type, hai thanh ghi rs và rt được đọc từ register file. ALUSrc = 0 khiến toán hạng B lấy từ rt. RegDst = 1 chọn rd làm thanh ghi đích. ALUOp = 10 yêu cầu ALU decoder xét funct để chọn phép toán. RegWrite = 1 và MemToReg = 0 khiến kết quả ALU được ghi trở lại rd. PC mặc định tăng 4.

4.2. Lệnh addi

Lệnh addi đọc rs, mở rộng immediate 16 bit thành 32 bit, sau đó ALUSrc = 1 chọn immediate làm toán hạng B. ALUOp = 00 chọn phép cộng. RegDst = 0 chọn rt làm thanh ghi đích; MemToReg = 0 chọn kết quả ALU; RegWrite = 1 cho phép ghi. Về mặt dữ liệu, phép tính là R[rt] = R[rs] + SignExt(imm).

4.3. Lệnh lw

Lệnh lw tính địa chỉ hiệu dụng bằng cách cộng giá trị rs với immediate đã mở rộng dấu. ALUSrc = 1 và ALUOp = 00. Địa chỉ do ALU tạo ra được đưa ra cổng aluresult để kết nối với data memory. Khi bộ nhớ trả readdata, MemToReg = 1 chọn dữ liệu bộ nhớ; RegDst = 0 chọn rt; RegWrite = 1 ghi dữ liệu vào rt. Công thức: R[rt] = Mem[R[rs] + SignExt(imm)].

Do dự án chưa chứa module data memory, hành vi đọc bộ nhớ mới được thể hiện qua cổng readdata chứ chưa được kiểm chứng bằng bộ nhớ thực trong mô phỏng.

4.4. Lệnh sw

Lệnh sw sử dụng rs làm thanh ghi cơ sở và immediate làm độ lệch. ALU tính địa chỉ hiệu dụng R[rs] + SignExt(imm). Dữ liệu cần ghi là giá trị rt, được đưa ra cổng writedata. MemWrite = 1 báo hiệu bộ nhớ dữ liệu thực hiện ghi tại địa chỉ aluresult. RegWrite = 0 vì lệnh không cập nhật register file.

4.5. Lệnh beq

Lệnh beq đọc rs và rt, ALUSrc = 0 để ALU nhận cả hai toán hạng từ register file. ALUOp = 01 làm ALU thực hiện phép trừ. Nếu rs = rt thì kết quả bằng 0 và Zero = 1. Main Control đặt Branch = 1; tại cấp tích hợp, pcsrc cần được tính bằng Branch AND Zero. Địa chỉ đích bằng PC + 4 + (SignExt(imm) << 2).

4.6. Lệnh j

Lệnh j không cần ghi thanh ghi hay truy cập dữ liệu. Jump = 1 điều khiển mux PC chọn pcjump. Địa chỉ jump được tạo bằng cách ghép 4 bit cao của PC+4, 26 bit đích trong lệnh và hai bit 0 ở cuối. Cách ghép này phù hợp với định dạng địa chỉ jump MIPS trong vùng 256 MB tương ứng.

4.7. Bảng tổng hợp đường xử lý

Lệnh	Nguồn ALU B	Phép ALU	Đích ghi	Nguồn write-back	Cập nhật PC
R-type	rt	Theo funct	rd	ALU	PC+4
addi	SignExt(imm)	ADD	rt	ALU	PC+4
lw	SignExt(imm)	ADD địa chỉ	rt	Data memory	PC+4
sw	SignExt(imm)	ADD địa chỉ	Không ghi	Không áp dụng	PC+4
beq	rt	SUB	Không ghi	Không áp dụng	PC+4 hoặc PCBranch
j	Không cần	Không cần	Không ghi	Không áp dụng	PCJump

5. THIẾT KẾ VÀ ĐÁNH GIÁ KIỂM THỬ

5.1. Môi trường và phương pháp kiểm thử

Các testbench sử dụng Verilog test fixture, tạo stimulus theo thời gian và xuất waveform VCD thông qua dumpfile và dumpvars. Tệp VCD có thể được mở bằng GTKWave để quan sát tín hiệu theo thời gian. Mã nguồn không ghi rõ phiên bản simulator hoặc log thực thi, vì vậy báo cáo không khẳng định đã chạy thành công trên một công cụ cụ thể.

5.2. Kiểm thử Main Control

tb_main_control lần lượt đưa sáu opcode R-type, lw, sw, beq, addi và j vào đầu vào op. Sau mỗi khoảng 10 ns, testbench so sánh các tín hiệu điều khiển với vector mong đợi bằng toán tử case equality === và in PASS/FAIL. Đây là cách kiểm tra có đối chiếu expected value, phù hợp để xác minh bảng giải mã opcode.

Giới hạn: testbench không có fatal hoặc cơ chế tổng hợp số lỗi; cuối test vẫn in thông báo hoàn thành và finish kể cả khi có trường hợp FAIL. Vì vậy, khi chạy tự động, cần bổ sung bộ đếm lỗi và kết thúc với trạng thái thất bại nếu có assertion không đạt.

5.3. Kiểm thử các khối hỗ trợ

tb_helpers kiểm tra MUX 2:1, adder, sign extension, shift-left 2 và flopr. Các stimulus tiêu biểu gồm chọn d0/d1 của mux; cộng 100 + 4; mở rộng dấu số dương +10 và số âm -12; dịch trái 2 bit giá trị 3 thành 12; và kiểm tra thanh ghi chốt dữ liệu tại cạnh lên clock.

Các giá trị kỳ vọng được ghi trong comment, nhưng testbench chưa dùng assertion để so sánh tự động. Do đó thông báo cuối 'tất cả các test ... hoàn tất thành công' chỉ cho biết luồng stimulus đã chạy đến cuối, không tự chứng minh từng đầu ra đúng. Cần chuyển comment kỳ vọng thành if/assert.

5.4. Kiểm thử Datapath

tb_datapath khởi tạo clock chu kỳ 10 ns, reset PC, kiểm tra PC tăng tuần tự theo các chu kỳ, sau đó mô phỏng hai lệnh addi: ghi giá trị 25 vào thanh ghi 1 và cộng tiếp 15 để ghi vào thanh ghi 2. Testbench hiện chưa đọc, so sánh trực tiếp giá trị trong register file; không có assertion xác nhận $1 = 25 hoặc $2 = 40. Ngoài ra, testbench chưa kiểm tra lw, sw, beq, j, các phép R-type hay tương tác với memory.

5.5. Ma trận kiểm thử đề xuất

Khối	**Ca kiểm thử cần có**	**Kết quả mong đợi**
PC / reset	Kích hoạt reset bất đồng bộ; nhả reset; nhiều cạnh clock	PC về 0; sau đó cập nhật đúng pcnext tại cạnh lên.
MUX	s=0 và s=1; dữ liệu khác nhau; kiểm tra nhiều độ rộng	Đầu ra lần lượt bằng d0 và d1.
Sign extension	0x0000, 0x7FFF, 0x8000, 0xFFFF	Mở rộng dấu đúng 32 bit.
ALU	AND, OR, ADD, SUB, SLT; số âm và biên	Kết quả và Zero đúng.
Register file	Đọc hai cổng; ghi tại cạnh lên; đọc zero; thử ghi zero	Ghi đồng bộ; đọc bất đồng bộ; zero luôn 0.
Main Control	Sáu opcode hỗ trợ và opcode không hợp lệ	Vector điều khiển khớp bảng chân trị.
ALU Decoder	ALUOp 00/01/10 và các funct hỗ trợ	Mã ALUControl đúng.
Datapath addi/R-type	Ghi và đọc lại thanh ghi đích	Giá trị write-back chính xác.
Datapath lw/sw	Ghi và đọc data memory giả lập	Địa chỉ và dữ liệu bộ nhớ đúng.
Datapath beq/j	Nhánh taken/not-taken; jump target	PC chọn đúng PC+4, PCBranch hoặc PCJump.

5.6. Tiêu chí đánh giá

Một test chỉ được xem là đạt khi có stimulus xác định, giá trị mong đợi độc lập, phép so sánh tự động và báo lỗi rõ ràng nếu sai. Nên bổ sung kiểm thử các giá trị biên, các trường hợp âm/dương, reset, lệnh liên tiếp và các tổ hợp điều khiển không hợp lệ. Với datapath, nên kiểm tra cả đầu ra trung gian (ALUResult, Zero, writedata, PC) lẫn trạng thái lưu trữ (register file và memory).

6. ĐÁNH GIÁ THIẾT KẾ

6.1. Ưu điểm

Phân chia thiết kế thành các mô-đun có chức năng tương đối độc lập, thuận lợi cho kiểm thử và tái sử dụng.

Sử dụng tham số WIDTH cho flopr và mux2, giúp các khối có tính tổng quát.

Datapath thể hiện được các đường xử lý trọng yếu: PC tuần tự, nhánh, jump, đọc thanh ghi, ALU, sign extension và write-back.

Main Control có nhánh default đưa tín hiệu về 0, hạn chế latch trong mô tả logic tổ hợp.

ALU hỗ trợ các phép toán cơ bản cần thiết cho nhóm lệnh đã công bố; phép SLT có xét kiểu số có dấu.

Có testbench tách riêng cho control unit, helper modules và datapath, tạo nền tảng cho quy trình verification.

6.2. Hạn chế và vấn đề kỹ thuật cần xử lý

Vấn đề quan sát từ mã nguồn	**Hướng xử lý**
Chưa có top-level CPU hoàn chỉnh; mips_top.v chỉ là đoạn instantiate ALU, không có module header/body đầy đủ.	Chưa thể mô phỏng toàn bộ CPU đầu-cuối. Cần xây dựng cpu_top kết nối main_control, ALU decoder, datapath và memory.
Chưa có instruction memory và data memory trong gói mã nguồn.	Chưa thể chứng minh luồng fetch và load/store hoàn chỉnh. Bổ sung ROM/RAM hoặc mô hình memory trong testbench.
pcsrc là đầu vào datapath nhưng chưa có logic Branch AND Zero trong hệ thống được cung cấp.	Lệnh beq chưa được tích hợp tự động; bổ sung pcsrc = branch & zero ở top-level.
tb_helpers không so sánh tự động các kết quả; tb_datapath cũng không kiểm tra giá trị ghi vào thanh ghi.	Thông báo hoàn tất không đồng nghĩa kiểm thử PASS. Bổ sung assertion/scoreboard.
Không có testbench riêng cho ALU và ALU decoder.	Các phép toán và giải mã funct chưa được xác minh độc lập.
Register file không reset/khởi tạo rf[1..31], và vẫn cho phép ghi rf[0].	Có thể phát sinh X khi đọc thanh ghi chưa được ghi. Nên khởi tạo trong testbench và chặn ghi vào địa chỉ 0.
ALU decoder dùng các mẫu casez có vùng chồng lấn tại	Nên dùng các giá trị chính xác hoặc quy định rõ mã không dùng.

Vấn đề quan sát từ mã nguồn	Hướng xử lý
ALUOp=11.	
Tên tệp alu_decorder.v sai chính tả so với alu_decoder.	Không ảnh hưởng logic nếu file list đúng, nhưng dễ gây nhầm khi tích hợp/build.
Có hai phiên bản ALU ở hai thư mục với tên cổng khác nhau (a/b/result và SrcA/SrcB/ALUResult).	Cần thống nhất một phiên bản hoặc quy định rõ module path để tránh trùng tên module khi biên dịch toàn bộ.
Testbench datapath chưa kiểm tra lệnh sw, lw, beq, j và R-type.	Độ bao phủ chức năng còn thấp; cần mở rộng bộ test.

7. ĐỀ XUẤT HOÀN THIỆN VÀ PHÁT TRIỂN

7.1. Hoàn thiện top-level CPU

Xây dựng module cpu_top có các cổng clk và reset. Bên trong, kết nối PC/datapath với instruction memory; nối opcode instr[31:26] vào main_control; nối ALUOp và funct instr[5:0] vào ALU decoder; nối các tín hiệu điều khiển về datapath; kết nối aluresult, writedata, readdata và mem_write với data memory. Tạo logic pcsrc = branch & zero. Đảm bảo tín hiệu jump có ưu tiên lựa chọn PCJump như thiết kế mux hiện tại.

7.2. Bổ sung mô hình bộ nhớ

Instruction memory có thể được mô hình hóa bằng mảng word 32 bit, địa chỉ lấy từ PC và dữ liệu instr được đọc tổ hợp. Data memory có thể dùng mảng word, hỗ trợ đọc tổ hợp và ghi đồng bộ theo cạnh lên khi MemWrite = 1. Cần thống nhất cách ánh xạ địa chỉ byte sang chỉ số word, ví dụ sử dụng địa chỉ [31:2] nếu memory căn hàng 4 byte.

7.3. Chuẩn hóa register file

Có thể thêm reset đồng bộ/bất đồng bộ cho các thanh ghi hoặc khởi tạo chúng trong testbench. Trong phần ghi, nên dùng điều kiện if (we3 && (a3 != 0)) để không ghi vào zero. Nếu lựa chọn không reset register file theo đặc tả kiến trúc, testbench cần chủ động nạp dữ liệu ban đầu.

7.4. Nâng cấp verification

Viết testbench riêng cho ALU, kiểm tra từng mã ALUControl và cờ Zero.

Viết testbench riêng cho ALU decoder, bao phủ từng funct và opcode ALUOp hợp lệ.
Thêm assertion cho helper modules thay vì chỉ ghi expected value trong comment.
Trong tb_datapath, kiểm tra PC, ALUResult, writedata và giá trị thanh ghi sau mỗi lệnh.
Xây dựng chương trình kiểm thử ngắn gồm nhiều lệnh liên tiếp, sau đó so sánh trạng thái cuối với kết quả tham chiếu tính bằng phần mềm.
Đưa kiểm thử vào script tự động, trả exit code khác 0 khi có lỗi.

7.5. Mở rộng tập lệnh

Sau khi hoàn thiện sáu nhóm lệnh cơ bản, có thể mở rộng các phép toán ALU và control cho XOR, NOR, shift, branch-not-equal hoặc các lệnh số học bổ sung. Mỗi lệnh mới cần được bổ sung đồng bộ ở bảng opcode, main control, ALU, datapath và testbench.

8. KẾT LUẬN

Dự án đã hiện thực được nhiều thành phần cốt lõi của datapath MIPS 32-bit đơn chu kỳ bằng Verilog, bao gồm PC, bộ cộng, mux, sign extension, shift-left 2, register file, ALU, main control và ALU decoder. Cấu trúc mô-đun giúp làm rõ vai trò của từng khối và thể hiện được các nguyên tắc cơ bản trong thiết kế bộ xử lý: tách biệt đường dữ liệu với điều khiển, giải mã lệnh theo opcode, lựa chọn toán hạng qua mux và cập nhật trạng thái tại cạnh clock.

Các testbench hiện có cung cấp bước kiểm tra ban đầu cho bộ điều khiển, các khối hỗ trợ và một kịch bản datapath đơn giản. Dù vậy, mức độ kiểm chứng còn hạn chế do một số test chưa có assertion, chưa bao phủ toàn bộ tập lệnh và chưa có hệ thống CPU tích hợp cùng bộ nhớ. Do đó, kết luận phù hợp nhất ở thời điểm này là dự án đã xây dựng nền tảng các khối chức năng của CPU MIPS đơn chu kỳ; để khẳng định hoạt động hoàn chỉnh, cần hoàn thiện top-level, memory, logic nhánh và kiểm thử đầu-cuối.

Qua quá trình thực hiện, đề tài giúp củng cố kiến thức về kiến trúc máy tính, thiết kế logic tổ hợp và tuần tự, mô tả phần cứng bằng Verilog, cũng như quy trình mô phỏng và xác minh thiết kế số.

TÀI LIỆU THAM KHẢO

[1] D. A. Patterson and J. L. Hennessy, Computer Organization and Design: The Hardware/Software Interface, 5th Edition, Morgan Kaufmann, 2014.

[2] D. A. Patterson and J. L. Hennessy, Computer Organization and Design RISC-V Edition / các tài liệu về datapath và control unit trong giáo trình tổ chức máy tính. (Chỉ sử dụng phần nguyên lý tương đồng; thiết kế trong báo cáo là MIPS).

[3] IEEE, IEEE Standard for Verilog Hardware Description Language, IEEE Std 1364-2005.

[4] MIPS Technologies, MIPS32 Architecture for Programmers, Volume II: The MIPS32 Instruction Set, tài liệu đặc tả tập lệnh MIPS32.

[5] Mã nguồn dự án Single-Cycle-Core-Hus do người học cung cấp: các tệp Verilog trong thư mục Main Modules Designs.

PHỤ LỤC A. DANH MỤC TỆP MÃ NGUỒN ĐÃ KHẢO SÁT

Tệp	**Vai trò**
Datapath_Interconnect/datapath.v	Kết nối các khối đường đi dữ liệu, PC logic, register file và ALU.
Datapath_Interconnect/alu.v	ALU 32 bit với AND, OR, ADD, SUB, SLT và cờ Zero.
Datapath_Interconnect/regfile.v	Register file 32 × 32 bit, hai cổng đọc và một cổng ghi.
Datapath_Helpers/adder.v	Bộ cộng 32 bit.
Datapath_Helpers/flopr.v	Thanh ghi tổng quát có reset bất đồng bộ mức cao.
Datapath_Helpers/mux2.v	MUX 2:1 có tham số độ rộng.
Datapath_Helpers/signext.v	Mở rộng dấu 16 bit lên 32 bit.
Datapath_Helpers/sl2.v	Dịch trái 2 bit.
Main_control_unit/main_control.v	Giải mã opcode và phát sinh tín hiệu điều khiển.
Alu Core/alu_decorder.v	Giải mã ALUOp và funct thành ALUControl.
tb_helpers.v; tb_datapath.v; tb_main_control.v	Các testbench kiểm thử khối hỗ trợ, datapath và main control.
Alu Core/mips_top.v	Chỉ chứa đoạn instantiate ALU; chưa phải top-level hoàn chỉnh.